

Topic 15: Building Prompt-Powered Applications

15.1 Architecture of a Prompt-Powered App

- Frontend: user interface that collects input and displays model output
- Prompt layer: templates that combine user input with system instructions
- LLM API: the model provider (OpenAI, Anthropic, Google, self-hosted)
- Output parser: code that extracts structured data from model output
- Validation layer: checks output quality and safety before displaying to user
- Feedback loop: collect user ratings to improve prompts over time

```
User Input
  |
  v
Input Validation (sanitise, length check)
  |
  v
Prompt Template (inject user input + system context)
  |
  v
LLM API Call (with retry + timeout)
  |
  v
Output Parser (extract JSON / text / code)
  |
  v
Output Validation (safety check, schema check)
  |
  v
Display to User
```

15.2 Prompt Templates

- A prompt template is a reusable prompt with variable placeholders
- Store templates separately from code -- in a database, file, or CMS for easy editing
- Version control templates: each change should be tracked, reviewed, and tested
- Use a templating engine (Jinja2, Handlebars) for complex conditional logic in prompts

```
# Python example using f-string template

def build_summary_prompt(article: str, audience: str, max_words: int) -> str:
    return f'''
You are a professional content editor.
Summarise the article below for a {audience} audience.
Length: under {max_words} words.
Format: 3 bullet points.
Do not include any preamble or explanation.

Article:
'''

{article}
'''
```

15.3 Handling Errors and Edge Cases

- Always implement retry logic with exponential backoff for API rate limit errors
- Set a maximum token limit on inputs to prevent runaway costs
- Fallback: if model output fails validation, retry once with a more constrained prompt
- Timeout: set a hard timeout on API calls -- never let the user wait indefinitely
- Logging: log every prompt and response (with PII removed) for debugging and auditing

```
import time

def call_llm_with_retry(prompt, max_retries=3):
    for attempt in range(max_retries):
        try:
            response = llm_api.complete(prompt, timeout=30)
            if is_valid(response):
                return response
            # Invalid output -- retry with tighter constraints
        except RateLimitError:
            time.sleep(2 ** attempt) # exponential backoff
        except TimeoutError:
            break
    return fallback_response()
```

15.4 Cost and Latency Optimisation

- Cache responses for identical or near-identical inputs -- LLM calls are expensive
- Use smaller/faster models for simple subtasks; large models only for complex reasoning

- Compress context: summarise long conversation history instead of passing it all each time
- Batch requests when processing many independent items simultaneously
- Monitor token usage per feature: identify which prompts are most expensive and optimise them

15.5 Prompt Engineering Best Practices Summary

- 1. Start simple: write the simplest prompt that could work, then add complexity only if needed
 - 2. Be explicit: state task, format, length, tone, constraints, and audience
 - 3. Use examples: few-shot beats zero-shot for non-standard tasks
 - 4. Separate concerns: use prompt chaining for complex multi-step tasks
 - 5. Test systematically: build a test set, measure quality, A/B test changes
 - 6. Secure your prompts: sanitise inputs, isolate untrusted content, validate outputs
 - 7. Version control everything: treat prompts like code -- review, test, document
 - 8. Iterate fast: prompt engineering is empirical -- experiment and measure
-